

Index

S.N	Lab Problems	Date	Signature
1	Implementation of Bubble, Selection and Insertion Sort	2082/02/15	
2	Implementation of Merge Sort	2082/02/15	
3	Implementation of Quick Sort	2082/02/15	
4	Implementation of Randomized Quick Sort	2082/02/16	
5	Implementation of 0/1 Knapsack problem using Dynamic Approach	2082/02/16	
6	Implementation of Matrix Chain Multiplication Problem	2082/02/16	
7	Implementation of String Editing	2082/02/18	
8	Implementation of Floyd Warshall Algorithm	2082/02/18	
9	Implementation of Dijkstra's single source shortest path	2082/02/18	
10	Implementation of Fractional Knapsack problem	2082/02/20	
11	Implementation of N Queen Problem using Backtracking	2082/02/20	
12	Kruskal's algorithm to find Minimum Spanning Tree of a given connected, Undirected graph	2082/02/20	
13	Program of Prim's Minimum Spanning Tree	2082/02/21	
14	Implementation of Subset Sum Problem	2082/02/21	
15	Implementation of Job Sequence in Deadlines	2082/02/21	

Lab 1: Implementation of Bubble , Selection and Insertion Sort

Bubble Sort:

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        // Last i elements are already in place
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
}
```

```

    bubbleSort(arr, n);

    printf("Sorted array using Bubble Sort:\n");

    printArray(arr, n);

    return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\bubbleSort.exe
Original array:
64 34 25 12 22 11 90
Sorted array using Bubble Sort:
11 12 22 25 34 64 90

```

Selection Sort:

```

#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        // Find the minimum element in the unsorted part
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }

        // Swap the found minimum with the first element
        if (minIndex != i) {
            temp = arr[i];

```

```

        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
}

```

```

void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

```

```

int main() {
    int arr[] = {29, 10, 14, 37, 13};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("Sorted array using Selection Sort:\n");
    printArray(arr, n);
    return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\selectionSort.exe
Original array:
29 10 14 37 13
Sorted array using Selection Sort:
10 13 14 29 37

```

Insertion Sort:

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

```
void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    insertionSort(arr, n);
    printf("Sorted array using Insertion Sort:\n");
}
```

```
    printArray(arr, n);  
    return 0;  
}
```

Output:

```
PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\insertionSort.exe  
Original array:  
12 11 13 5 6  
Sorted array using Insertion Sort:  
5 6 11 12 13
```

Lab 2: Implementation of Merge Sort

```
#include <stdio.h>  
  
void merge(int arr[], int left, int mid, int right) {  
    int i, j, k;  
    int n1 = mid - left + 1;  
    int n2 = right - mid;  
  
    // Create temp arrays  
    int L[n1], R[n2];  
  
    // Copy data to temp arrays  
    for (i = 0; i < n1; i++)  
        L[i] = arr[left + i];  
    for (j = 0; j < n2; j++)  
        R[j] = arr[mid + 1 + j];  
  
    // Merge temp arrays back into arr[]  
    i = 0;
```

```

j = 0;
k = left;

while (i < n1 && j < n2) {
    if (L[i] <= R[j])
        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}

// Copy any remaining elements
while (i < n1)
    arr[k++] = L[i++];
while (j < n2)
    arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

```

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
    printf("\n");  
}  
  
int main() {  
    int arr[] = {38, 27, 43, 3, 9, 82, 10};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    printf("Original array:\n");  
    printArray(arr, n);  
    mergeSort(arr, 0, n - 1);  
    printf("Sorted array using Merge Sort:\n");  
    printArray(arr, n);  
    return 0;  
}
```

Output:

```
PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\mergeSort.exe  
Original array:  
38 27 43 3 9 82 10  
Sorted array using Merge Sort:  
3 9 10 27 38 43 82
```


Lab 3: Implementation of Quick Sort

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // choosing first element as pivot
    int i = low + 1;
    int j = high;

    while (i <= j) {
        while (i <= high && arr[i] <= pivot)
            i++;
        while (j >= low && arr[j] > pivot)
            j--;
        if (i < j)
            swap(&arr[i], &arr[j]);
    }

    swap(&arr[low], &arr[j]); // place pivot in correct position
    return j;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
    }
}
```

```

        quickSort(arr, p + 1, high);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {24, 9, 29, 14, 19, 27};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Sorted array using Quick Sort:\n");
    printArray(arr, n);
    return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\quickSort.exe
Original array:
24 9 29 14 19 27
Sorted array using Quick Sort:
9 14 19 24 27 29

```

Lab 4: Implementation of Randomized Quick Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
// Swap two elements
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Partition function (Lomuto style)
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high]; // Pivot is at end
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return i + 1;
```

```
}
```

```
// Randomized partition function
```

```
int randomizedPartition(int arr[], int low, int high) {
```

```
    int randomIndex = low + rand() % (high - low + 1); // Random pivot
    swap(&arr[randomIndex], &arr[high]); // Swap with end
    return partition(arr, low, high);
}
```

// Randomized Quick Sort

```
void randomizedQuickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = randomizedPartition(arr, low, high);
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}
```

// Print array

```
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

// Main function

```
int main() {
    int arr[] = {23, 45, 12, 8, 19, 56, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
```

// Seed random number generator

```
srand(time(0));
printf("Original array:\n");
printArray(arr, n);
```

```
    randomizedQuickSort(arr, 0, n - 1);  
    printf("Sorted array using Randomized Quick Sort:\n");  
    printArray(arr, n);  
    return 0;  
}
```

Output:

```
PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\randomQuicSort.exe  
Original array:  
23 45 12 8 19 56 4  
Sorted array using Randomized Quick Sort:  
4 8 12 19 23 45 56
```

Lab 5: Implementation of 0/1 Knapsack problem using Dynamic Approach

```
#include <stdio.h>

#include <stdlib.h>

// Function to return max of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Knapsack function using Dynamic Programming
int knapsack(int capacity, int weight[], int value[], int n) {
    int i, w;
    int K[n + 1][capacity + 1];

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (weight[i - 1] <= w)
                K[i][w] = max(value[i - 1] + K[i - 1][w - weight[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    return K[n][capacity];
}

int main() {
    int n, capacity;
    printf("Enter number of items: ");
```

```

scanf("%d", &n);

int value[n], weight[n];

printf("Enter values of %d items:\n", n);
for (int i = 0; i < n; i++) {
    printf("Value of item %d: ", i + 1);
    scanf("%d", &value[i]);
}

printf("Enter weights of %d items:\n", n);
for (int i = 0; i < n; i++) {
    printf("Weight of item %d: ", i + 1);
    scanf("%d", &weight[i]);
}

printf("Enter capacity of knapsack: ");
scanf("%d", &capacity);

int maxProfit = knapsack(capacity, weight, value, n);

printf("\nMaximum value that can be obtained: %d\n", maxProfit);

return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\0by1KnapsackDynamic.exe
Enter number of items: 3
Enter values of 3 items:
Value of item 1: 60
Value of item 2: 100
Value of item 3: 120
Enter weights of 3 items:
Weight of item 1: 15
Weight of item 2: 20
Weight of item 3: 30
Enter capacity of knapsack: 50

Maximum value that can be obtained: 220

```

Lab 6: Implementation of Matrix Chain Multiplication problem

```
#include <stdio.h>

#include <limits.h>

// Function to find minimum number of multiplications
int matrixChainOrder(int dims[], int n) {

    int m[n][n]; // m[i][j] stores minimum multiplications from matrix i to j

    // Set diagonal to 0 (only one matrix, no multiplication)
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    // l is chain length
    for (int l = 2; l < n; l++) {
        for (int i = 1; i < n - l + 1; i++) {
            int j = i + l - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k + 1][j] + dims[i - 1] * dims[k] * dims[j];
                if (cost < m[i][j])
                    m[i][j] = cost;
            }
        }
    }

    return m[1][n - 1]; // minimum cost to multiply from M1 to Mn-1
}

int main() {
```



```

int n;

printf("Enter number of matrices: ");

scanf("%d", &n);

int dims[n + 1];

printf("Enter dimensions (P0 P1 ... Pn) such that matrix i has dimension Pi-1 x Pi:\n");

for (int i = 0; i <= n; i++) {

    printf("P%d: ", i);

    scanf("%d", &dims[i]);

}

int minMultiplications = matrixChainOrder(dims, n + 1);

printf("\nMinimum number of multiplications: %d\n", minMultiplications);

return 0;

}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\matrixChainMultiplication.exe
Enter number of matrices: 4
Enter dimensions (P0 P1 ... Pn) such that matrix i has dimension Pi-1 x Pi:
P0: 10
P1: 20
P2: 30
P3: 30
P4: 50

Minimum number of multiplications: 30000

```

dp[i - 1][j], // Remove

```

        dp[i - 1][j - 1] // Replace
    );
}
}
return dp[m][n];
}

int main() {
    char str1[100], str2[100];
    printf("Enter the first string: ");
    scanf("%s", str1);
    printf("Enter the second string: ");
    scanf("%s", str2);

    int distance = editDistance(str1, str2);
    printf("\nMinimum Edit Distance = %d\n", distance);
    return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\stringEditing.exe
Enter the first string: sunday
Enter the second string: saturday

Minimum Edit Distance = 3

```

Lab 8: Implementation of Floyd Warshall Algorithm

```
#include <stdio.h>
```

```
#define INF 99999
```

```
#define V 4
```

```
void printSolution(int dist[V][V]);
```

```
void floydWarshall(int graph[V][V]) {
```

```
    int dist[V][V];
```

```
    int steps = 0; // Step counter
```

```
    // Initialize the distance matrix
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            dist[i][j] = graph[i][j];
```

```
            steps++; // Count initialization step
```

```
        }
```

```
    }
```

```
    // Floyd-Warshall Algorithm
```

```
    for (int k = 0; k < V; k++) {
```

```
        for (int i = 0; i < V; i++) {
```

```
            for (int j = 0; j < V; j++) {
```

```
                steps++; // Step for comparison
```

```
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
```

```
                    dist[i][j] = dist[i][k] + dist[k][j];
```

```
                    steps++; // Step for update
```

```
                }
```

```
            }
```

```
        }
```

```

    }

    // Print the shortest distance matrix
    printSolution(dist);

    // Print total number of steps
    printf("\nTotal Steps Executed: %d\n", steps);
}

void printSolution(int dist[V][V]) {
    printf("Shortest distances between every pair of vertices:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int graph[V][V] = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {5, INF, 0, 1},
        {INF, INF, 3, 0}
    };

```

```
floydWarshall(graph);  
return 0;  
}
```

Output:

```
PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\flyordwarshall.exe  
Shortest distances between every pair of vertices:  
    0     5     8     9  
    8     0     3     4  
    5    10     0     1  
    8    13     3     0  
  
Total Steps Executed: 87
```

Lab 9: Implementation of Dijkstra's single source shortest path

```
#include <stdio.h>

#include <limits.h>

#define V 5 // Number of vertices in the graph
#define INF 99999

// Find the vertex with minimum distance value, from the set of vertices not yet processed
int minDistance(int dist[], int visited[]) {
    int min = INF, min_index = -1;
    for (int v = 0; v < V; v++) {
        if (visited[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Print the distance array
void printSolution(int dist[], int src) {
    printf("Vertex\t\tDistance from Source (%d)\n", src);
    for (int i = 0; i < V; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

// Dijkstra's algorithm implementation
void dijkstra(int graph[V][V], int src) {
    int dist[V]; // Output array: dist[i] will hold the shortest distance from src to i
```

```

int visited[V]; // visited[i] will be true if vertex i is included in shortest path tree

// Initialization
for (int i = 0; i < V; i++) {
    dist[i] = INF;
    visited[i] = 0;
}
dist[src] = 0; // Distance from source to itself is always 0

// Find shortest path for all vertices
for (int count = 0; count < V - 1; count++) {
    int u = minDistance(dist, visited);
    visited[u] = 1;

    // Update dist[v] only if not visited and path through u is shorter
    for (int v = 0; v < V; v++) {
        if (!visited[v] && graph[u][v] && dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

// Print the result
printSolution(dist, src);
}

// Main function
int main() {
    // Example graph represented as adjacency matrix

```



```

int graph[V][V] = {
    {0, 10, 0, 30, 100},
    {0, 0, 50, 0, 0},
    {0, 0, 0, 0, 10},
    {0, 0, 20, 0, 60},
    {0, 0, 0, 0, 0}
};

int source;

printf("Enter the source vertex (0 to %d): ", V - 1);
scanf("%d", &source);

dijkstra(graph, source);

return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\dijkstraShortestPath.exe
Enter the source vertex (0 to 4): 0
Vertex      Distance from Source (0)
0           0
1           10
2           50
3           30
4           60

```

Lab 10: Implementation of Fractional Knapsack problem

```
#include <stdio.h>

struct Item {
    int weight;
    int value;
};

// Function to swap two items
void swap(struct Item *a, struct Item *b) {
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}

// Function to sort items by value/weight ratio in descending order
void sortItems(struct Item arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            float r1 = (float)arr[j].value / arr[j].weight;
            float r2 = (float)arr[j + 1].value / arr[j + 1].weight;
            if (r1 < r2) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

// Function to perform Fractional Knapsack
float fractionalKnapsack(struct Item items[], int n, int capacity) {
    sortItems(items, n);
```

```

float totalValue = 0.0;

for (int i = 0; i < n && capacity > 0; i++) {
    if (items[i].weight <= capacity) {
        // Take the full item
        capacity -= items[i].weight;
        totalValue += items[i].value;
    } else {
        // Take the fractional part
        float fraction = (float)capacity / items[i].weight;
        totalValue += items[i].value * fraction;
        break; // knapsack is full
    }
}

return totalValue;
}

int main() {
    int n, capacity;

    printf("Enter the number of items: ");
    scanf("%d", &n);

    struct Item items[n];

    printf("Enter weight and value of each item:\n");
    for (int i = 0; i < n; i++) {
        printf("Item %d - Weight: ", i + 1);

```

```

scanf("%d", &items[i].weight);

printf("Item %d - Value: ", i + 1);

scanf("%d", &items[i].value);
}

printf("Enter the capacity of the knapsack: ");

scanf("%d", &capacity);

float maxValue = fractionalKnapsack(items, n, capacity);

printf("\nMaximum value in knapsack = %.2f\n", maxValue);

return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\01FractionalKnapsack.exe
Enter the number of items: 3
Enter weight and value of each item:
Item 1 - Weight: 10
Item 1 - Value: 60
Item 2 - Weight: 20
Item 2 - Value: 100
Item 3 - Weight: 30
Item 3 - Value: 120
Enter the capacity of the knapsack: 50

Maximum value in knapsack = 240.00

```

Lab 11: Implementation of N Queen Problem using Backtracking

```
#include <stdio.h>

#define N 8 // Change this value to solve for different board sizes

int board[N];

// Function to print the board
void printSolution() {
    printf("One of the solutions:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
    printf("\n");
}

// Function to check if a queen can be placed at board[row][col]
int isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col || // Same column
            board[i] - i == col - row || // Same major diagonal
            board[i] + i == col + row) // Same minor diagonal
            return 0;
    }
    return 1;
}

// Recursive utility function to solve N-Queens
int solveNQueens(int row) {
```

```

if (row == N) {
    printSolution();
    return 1; // Return 1 to stop after first solution
}

for (int col = 0; col < N; col++) {
    if (isSafe(row, col)) {
        board[row] = col;
        if (solveNQueens(row + 1))
            return 1;
    }
}

return 0; // Backtrack
}

int main() {
    if (!solveNQueens(0)) {
        printf("No solution exists.\n");
    }
    return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\nQueenProblem.exe
One of the solutions:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .

```

Lab 12: Kruskal's algorithm to find Minimum Spanning Tree of a given connected, Undirected graph

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

// Structure to represent an edge
struct Edge {
    int src, dest, weight;
};

// Structure to represent a subset for union-find
struct Subset {
    int parent;
    int rank;
};

// Function to compare two edges (used in qsort)
int compareEdges(const void* a, const void* b) {
    struct Edge* e1 = (struct Edge*)a;
    struct Edge* e2 = (struct Edge*)b;
    return e1->weight - e2->weight;
}

// Find set of an element i (uses path compression)
int find(struct Subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

// Union of two sets x and y (uses union by rank)
```

```

void unionSets(struct Subset subsets[], int x, int y) {
    int rootX = find(subsets, x);
    int rootY = find(subsets, y);

    if (subsets[rootX].rank < subsets[rootY].rank) {
        subsets[rootX].parent = rootY;
    } else if (subsets[rootX].rank > subsets[rootY].rank) {
        subsets[rootY].parent = rootX;
    } else {
        subsets[rootY].parent = rootX;
        subsets[rootX].rank++;
    }
}

```

// Kruskal's algorithm

```

void kruskal(struct Edge edges[], int V, int E) {
    struct Edge result[MAX]; // Store MST edges
    int e = 0; // index for result[]
    int i = 0; // index for sorted edges

    // Sort edges by increasing weight
    qsort(edges, E, sizeof(edges[0]), compareEdges);

```

// Allocate memory for union-find subsets

```

struct Subset subsets[V];
for (int v = 0; v < V; v++) {
    subsets[v].parent = v;
    subsets[v].rank = 0;
}

```



```

// Process edges one by one
while (e < V - 1 && i < E) {
    struct Edge next = edges[i++];

    int x = find(subsets, next.src);
    int y = find(subsets, next.dest);

    // If including this edge doesn't cause a cycle, include it in MST
    if (x != y) {
        result[e++] = next;
        unionSets(subsets, x, y);
    }
}

// Print the result
printf("Edges in the Minimum Spanning Tree:\n");
int totalWeight = 0;
for (i = 0; i < e; i++) {
    printf("%d -- %d == %d\n", result[i].src, result[i].dest, result[i].weight);
    totalWeight += result[i].weight;
}
printf("Total weight of MST = %d\n", totalWeight);
}

int main() {
    int V, E;
    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);
    struct Edge edges[MAX];
    printf("Enter edges (source destination weight):\n");

```

```

for (int i = 0; i < E; i++) {
    printf("Edge %d: ", i + 1);
    scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);
}
kruskal(edges, V, E);
return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\kruskalMinSpanTree.exe
Enter number of vertices and edges: 4 5
Enter edges (source destination weight):
Edge 1: 0 1 10
Edge 2: 0 2 6
Edge 3: 0 3 5
Edge 4: 1 3 15
Edge 5: 2 3 4
Edges in the Minimum Spanning Tree:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Total weight of MST = 19

```

Lab 13: Implementation of Prim's Minimum Spanning Tree

```
#include <stdio.h>

#include <limits.h>

#define V 100 // Max number of vertices

int minKey(int key[], int mstSet[], int n) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < n; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}

void printMST(int parent[], int graph[V][V], int n) {
    int totalWeight = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < n; i++) {
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
        totalWeight += graph[i][parent[i]];
    }
    printf("Total weight of MST = %d\n", totalWeight);
}

void primMST(int graph[V][V], int n) {
    int parent[V]; // Stores MST
    int key[V];    // Key values
    int mstSet[V]; // Vertices included in MST

    for (int i = 0; i < n; i++)
        key[i] = INT_MAX, mstSet[i] = 0;
```

```

key[0] = 0;
parent[0] = -1;

for (int count = 0; count < n - 1; count++) {
    int u = minKey(key, mstSet, n);
    mstSet[u] = 1;

    for (int v = 0; v < n; v++)
        if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v])
            parent[v] = u, key[v] = graph[u][v];
}

printMST(parent, graph, n);
}

int main() {
    int n;
    int graph[V][V];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix (use 0 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    primMST(graph, n);

```

```
    return 0;  
}
```

Output:

```
PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\prismMinSpanTree.exe  
Enter number of vertices: 5  
Enter the adjacency matrix (use 0 for no edge):  
0 2 0 6 0  
2 0 3 8 5  
0 3 0 0 7  
6 8 0 0 9  
0 5 7 9 0  
Edge    Weight  
0 - 1    2  
1 - 2    3  
0 - 3    6  
1 - 4    5  
Total weight of MST = 16
```

Lab 14: Implementation of Subset Sum Problem

```
#include <stdio.h>

#define MAX 100

int subsetSum(int set[], int n, int sum) {
    int dp[n+1][sum+1];

    // Initialize dp[0][...] = 0 and dp[...][0] = 1
    for (int i = 0; i <= n; i++)
        dp[i][0] = 1;
    for (int j = 1; j <= sum; j++)
        dp[0][j] = 0;

    // Fill the dp table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= sum; j++) {
            if (set[i-1] > j)
                dp[i][j] = dp[i-1][j];
            else
                dp[i][j] = dp[i-1][j] || dp[i-1][j - set[i-1]];
        }
    }

    return dp[n][sum];
}

int main() {
    int set[MAX], n, sum;
```

```

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter the elements:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &set[i]);

printf("Enter the target sum: ");
scanf("%d", &sum);

if (subsetSum(set, n, sum))
    printf("Subset with given sum exists.\n");
else
    printf("No subset with given sum found.\n");

return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\subsetSumProblem.exe
Enter number of elements: 5
Enter the elements:
3 35 5 12 7
Enter the target sum: 10
Subset with given sum exists.

```

Lab 15: Implementation of Job Sequence in Deadlines

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
// Job structure
typedef struct {
    char id[5];
    int deadline;
    int profit;
} Job;

// Comparison function to sort jobs by profit (descending)
int compare(const void *a, const void *b) {
    Job *job1 = (Job *)a;
    Job *job2 = (Job *)b;
    return job2->profit - job1->profit;
}

int min(int a, int b) {
    return (a < b) ? a : b;
}

void jobSequencing(Job jobs[], int n) {
    qsort(jobs, n, sizeof(Job), compare); // Sort jobs by profit descending

    int maxDeadline = 0;
    for (int i = 0; i < n; i++)
        if (jobs[i].deadline > maxDeadline)
            maxDeadline = jobs[i].deadline;
```



```

char schedule[maxDeadline + 1][5]; // Store job ids
int slot[maxDeadline + 1];        // Track free slots

for (int i = 1; i <= maxDeadline; i++)
    slot[i] = 0;

int totalProfit = 0;

for (int i = 0; i < n; i++) {
    for (int j = min(maxDeadline, jobs[i].deadline); j >= 1; j--) {
        if (!slot[j]) {
            slot[j] = 1;
            strcpy(schedule[j], jobs[i].id);
            totalProfit += jobs[i].profit;
            break;
        }
    }
}

printf("\nSelected Jobs in sequence:\n");
for (int i = 1; i <= maxDeadline; i++) {
    if (slot[i])
        printf("Time Slot %d: Job %s\n", i, schedule[i]);
}

printf("Total Profit: %d\n", totalProfit);
}

int main() {

```

```

int n;
Job jobs[MAX];

printf("Enter number of jobs: ");
scanf("%d", &n);

printf("Enter job details (ID Deadline Profit):\n");
for (int i = 0; i < n; i++) {
    scanf("%s %d %d", jobs[i].id, &jobs[i].deadline, &jobs[i].profit);
}
jobSequencing(jobs, n);
return 0;
}

```

Output:

```

PS D:\Arjun Mijar(109) Lab Reports\Data Analysis and Algorithm> .\jobSequenceDeadlines.exe
Enter number of jobs: 5
Enter job details (ID Deadline Profit):
j1 2 100
j2 1 19
j3 3 26
j4 1 25
j5 3 15

Selected Jobs in sequence:
Time Slot 1: Job j4
Time Slot 2: Job j1
Time Slot 3: Job j3
Total Profit: 151

```